

# Bellman Ford Algorithm

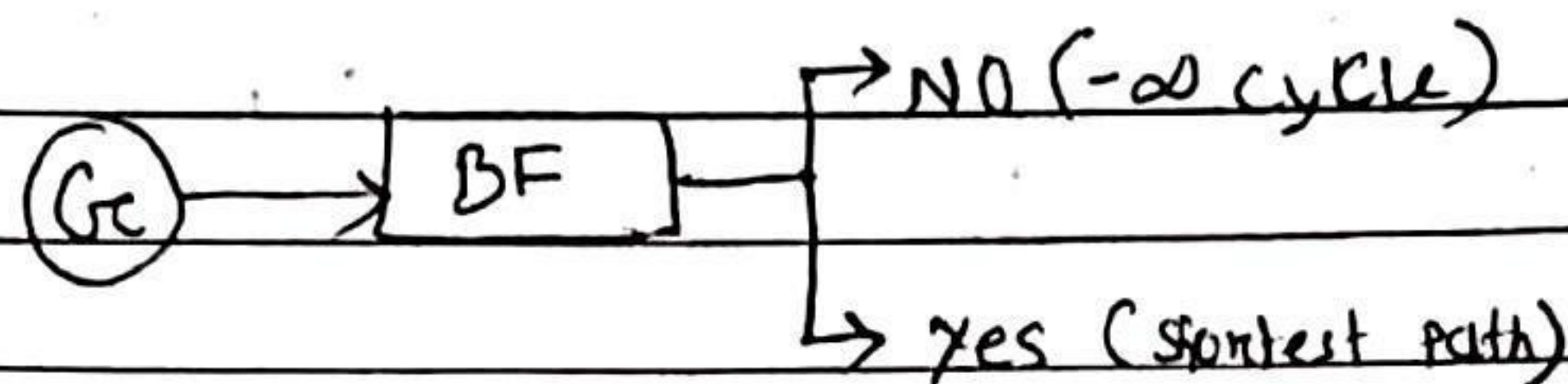
atlantis

Date \_\_\_\_\_

Page \_\_\_\_\_

## Bellman Ford Introduction =

Bellman Ford algo have capability -  
 To findout whether a graph is having negative  
 weight cycle or not.

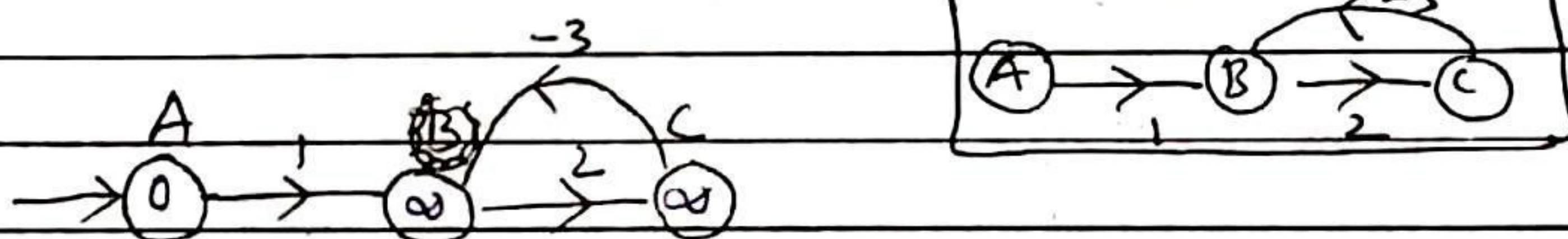


→ Bellman Ford algo slower than Dijkstra algorithm.

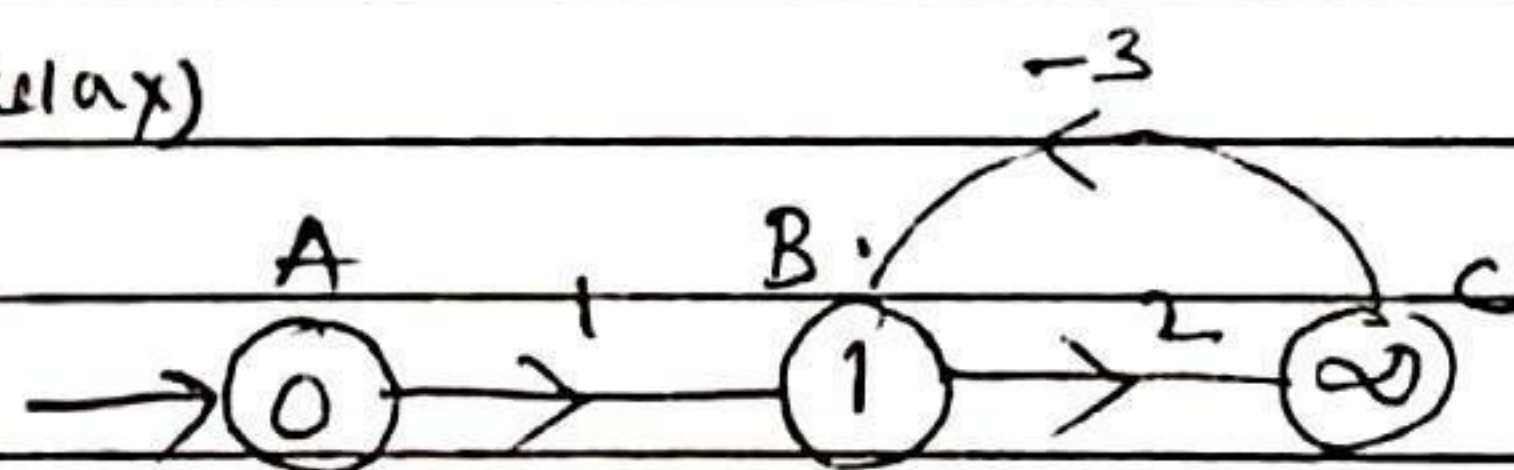
Bellman Ford works on this rule -

→ Shortest path in the graph will never contain more than  $(n-1)$  edges.

EX: Bellman Ford findout there is a Negative weight cycle -

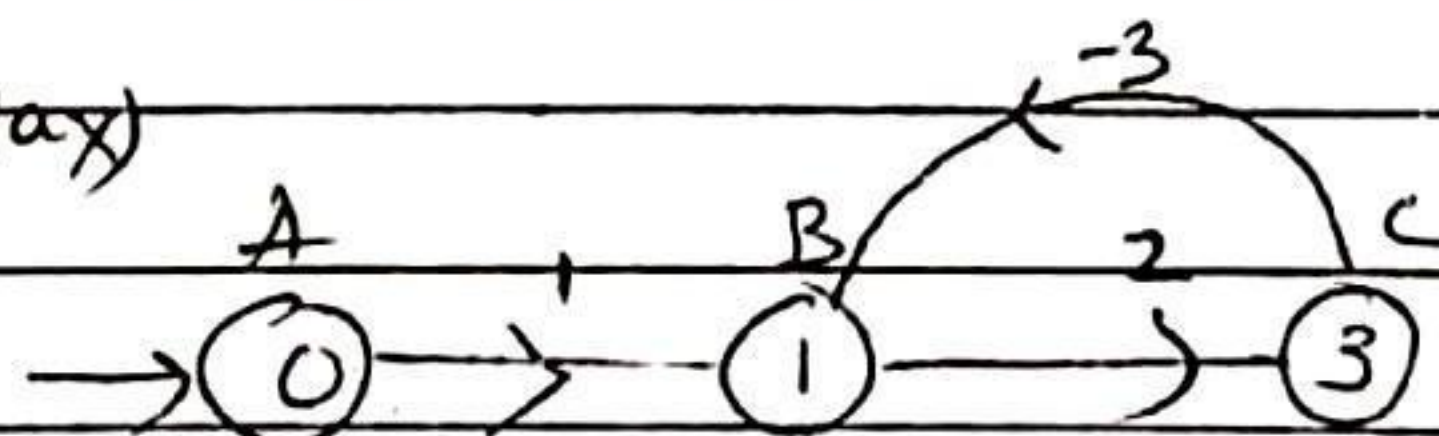


I. (relax)

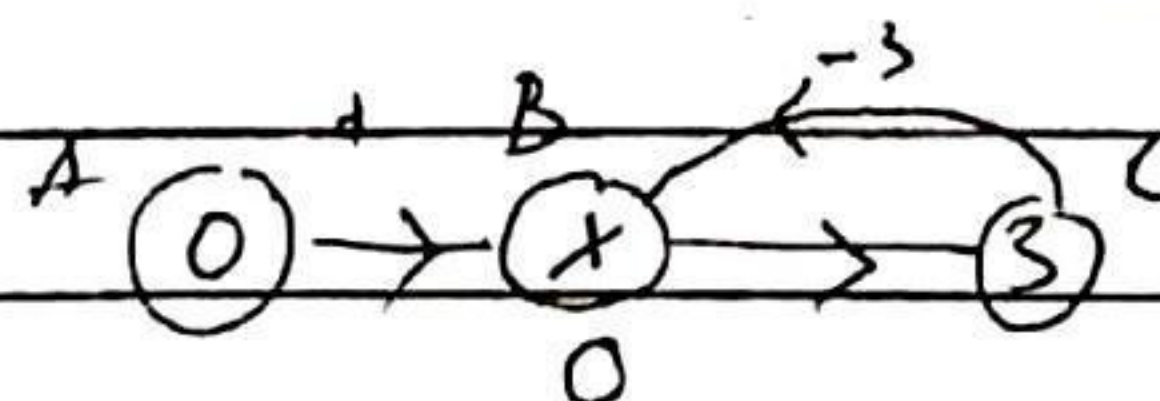


→ When no of vertices is 'n', then relaxation will do  $(n-1)$  time

II. (relax)



Stop.



Relaxing time =  $(V-1) + 1$

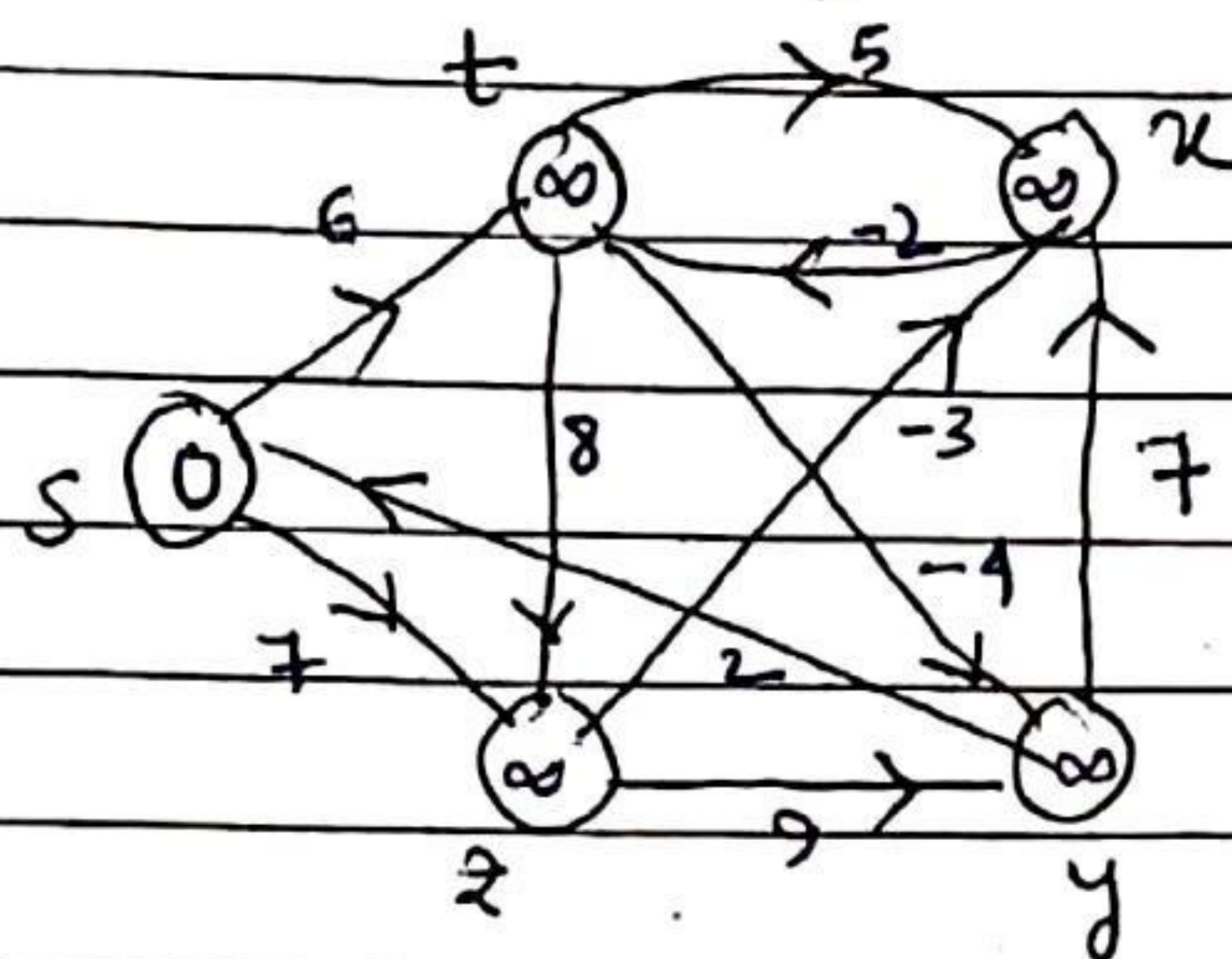
↓ =  $O(V)$

Relaxing Edges =  $O(VE)$

✓ Time complexity =  ~~$O(V^2)$~~  or  $O(VE)$ .

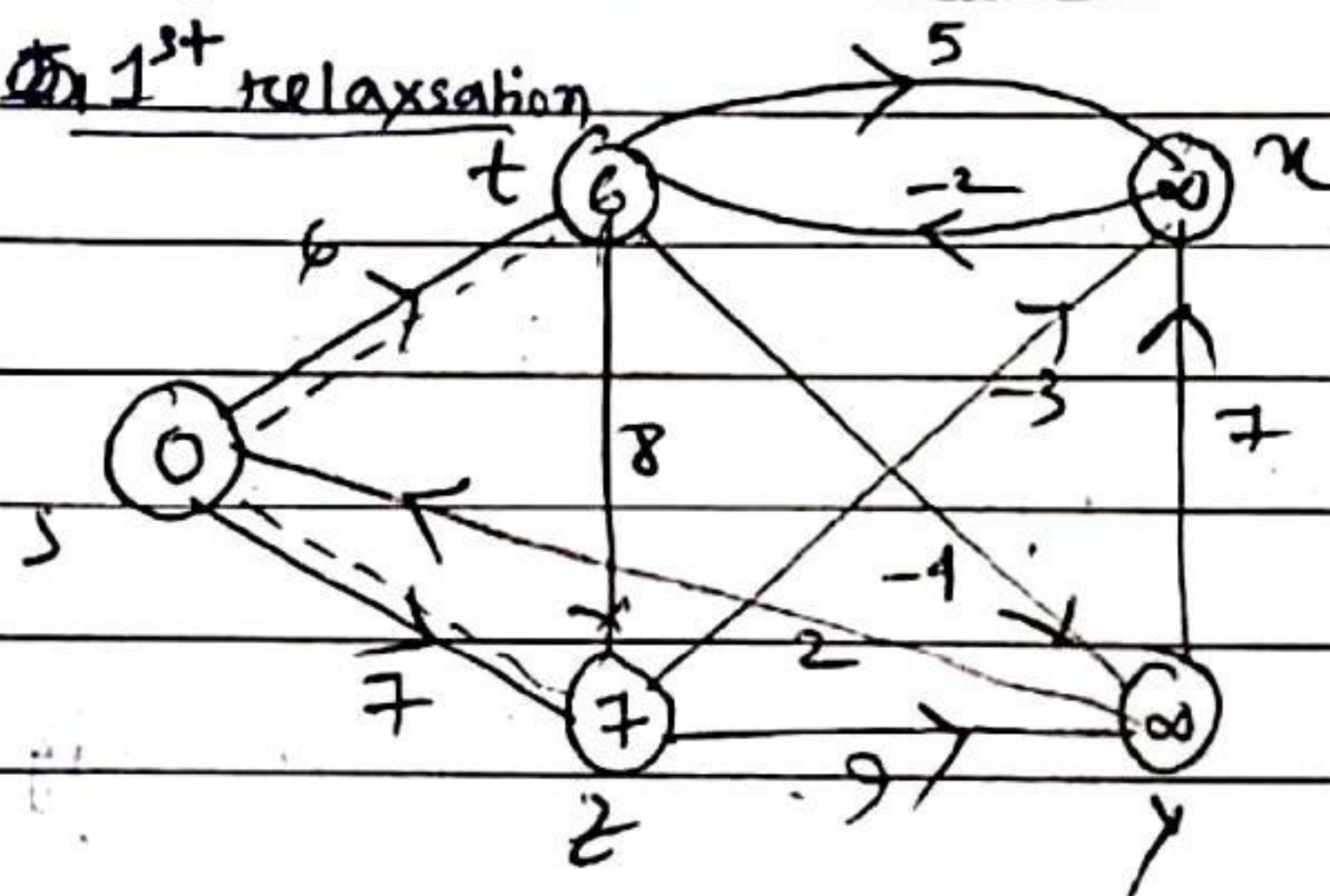
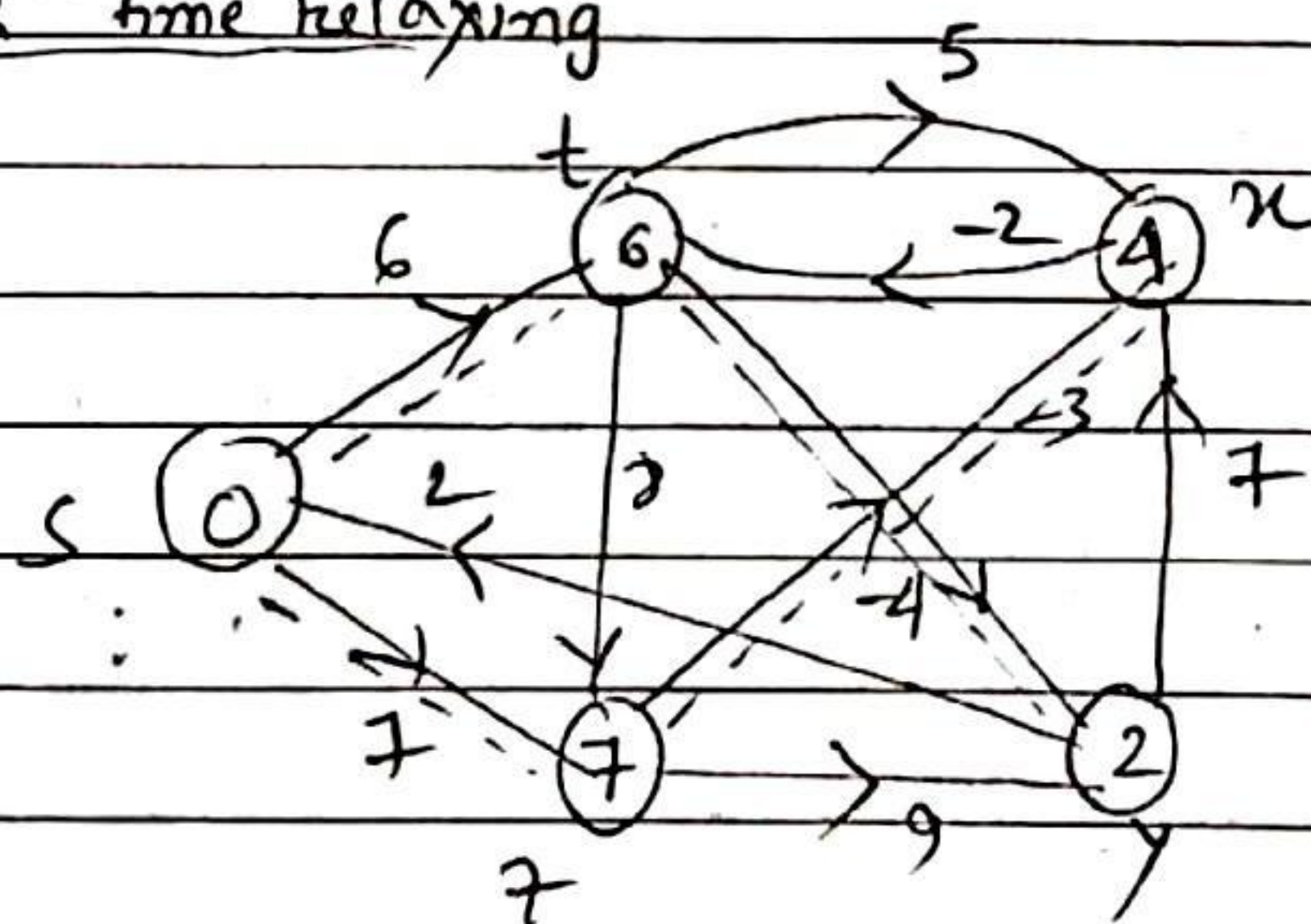
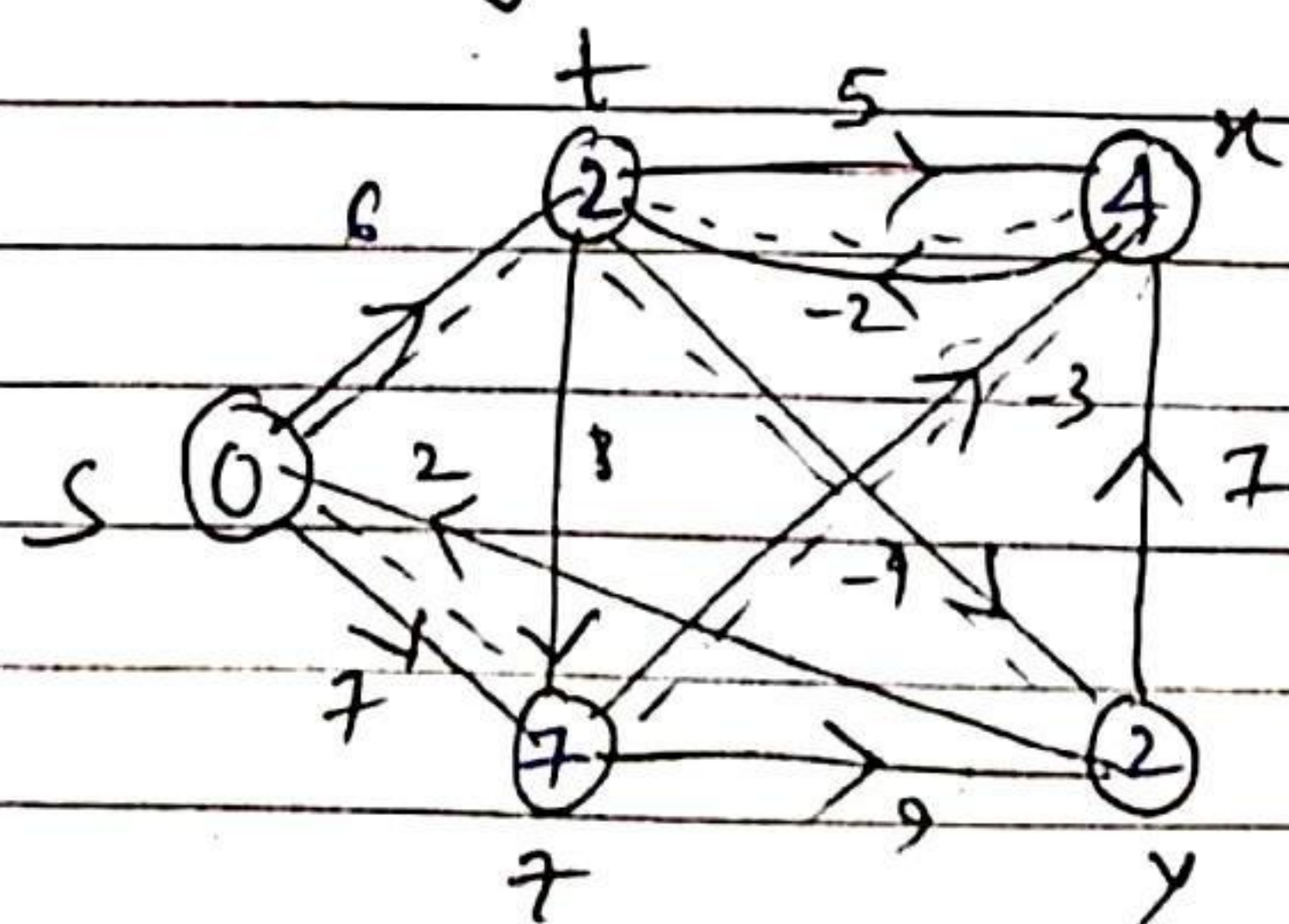
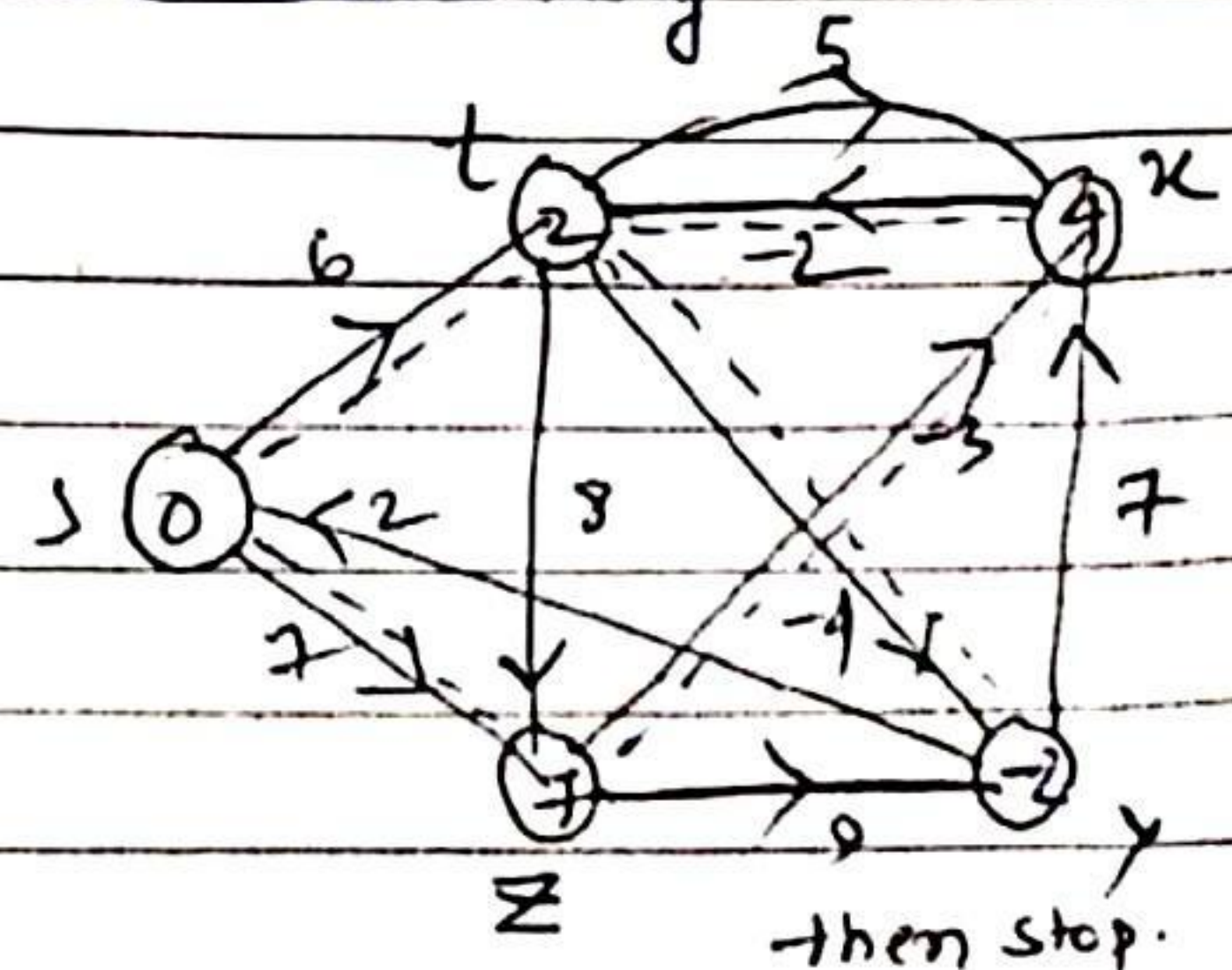


[Exo]

 $(t-x) (t-y) (t-z) (x-t) (y-x) (y-z) (y-x) (y-s)$ 
 $(s-t) (s-y)$ 


→ you can relax in any order.

→  $n \rightarrow (n-1)$  time relax (vertices)

1<sup>st</sup> relaxation2<sup>nd</sup> time relaxing3<sup>rd</sup> time relaxing4<sup>th</sup> time relaxing

then stop.



BELLMAN-FORD ( $G, w, s$ )

{

Initialize single source ( $G, s$ ) //  $O(V)$ for  $i = 1$  to  $|G.V| - 1$  //  $(V-1)$ for each edge  $(u, v) \in G.E$  //  $(E)$ RELAX  $(u, v, w)$  //  $O(1)$ 

$$\left. \begin{array}{l} (V-1)(E) O(1) \\ = O(VE) \end{array} \right\}$$
for ~~for~~ each edge  $(u, v) \in G.E$  //  $O(E)$ if  $(v.d > u.d + w(u, v))$ 

return FALSE

return TRUE

}

Time complexity =  $O(E \cdot V)$